

Customer Onboarding AI

Task 6A — ContextPayload Request Seam

Beginner-Friendly • Enterprise-Grade Implementation Guide

Status: Implementation completed • Human checkpoint pending

1. Purpose of Task 6A

Task 6A introduces the ContextPayload request seam into the Customer Onboarding AI application. The application now accepts an explicit customer-onboarding context through the HTTP API, carries that context into the Domain Runtime, uses it to control the developer-authored UISpec metadata, and passes the same context to the existing deterministic validator.

The important point is that Task 6A introduces contextual input without introducing a runtime AI model. The application remains deterministic at this stage.

Key lesson: Context is now explicit, validated, and traceable — but the system is still not probabilistic.

2. Where Task 6A Fits

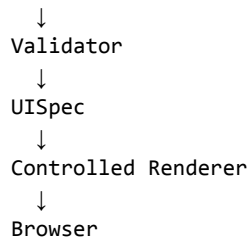
TASK 1	Project foundation	✓
TASK 2	Model contract + JSON Schema	✓
TASK 3	Deterministic validator	✓
TASK 4	Controlled UISpec renderer	✓
TASK 5	Runnable React UI	✓
TASK 6	FastAPI Domain Runtime	✓
TASK 6A	ContextPayload request seam	✓

NEXT	Task 6A checkpoint	
NEXT	AI client / model seam	
NEXT	Runtime Model Gateway	
NEXT	Probabilistic model → UISpec	
NEXT	AI evaluations + feedback loop	

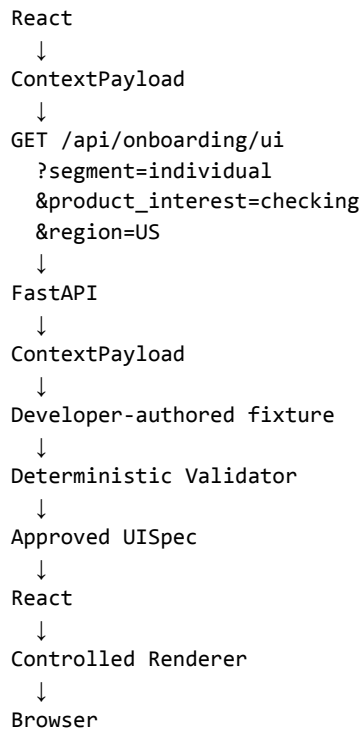
3. What Changed from Task 6 to Task 6A

Before Task 6A:

```
graph TD
    React --> GET[GET /api/onboarding/ui]
    GET --> FastAPI[FastAPI]
    FastAPI --> Fixture[Fixed developer-authored fixture]
```



After Task 6A:



The future runtime model will eventually occupy the generation seam after ContextPayload. That seam is intentionally not implemented in Task 6A.

4. What Is ContextPayload?

ContextPayload is the structured context supplied to the onboarding UI generation path. For Task 6A, the HTTP API exposes exactly three required fields.

Field	Allowed values	Purpose
segment	individual, small-business, corporate	Customer segment
product_interest	checking, savings, credit-card, loan, unspecified	Product context
region	US, EU, UK, APAC	Regional context

These values mirror the existing model contract. Task 6A deliberately does not broaden the accepted set.

5. Python ContextPayload Design

The implementation adds `api/context.py`. It defines three string Enum classes matching the contract and a frozen `ContextPayload` dataclass. The dataclass exposes helpers for converting the context into the dictionary shapes required by the existing service and validator.

- The dataclass is frozen, preventing accidental mutation after construction.
- The three API-visible fields are enum-typed.
- `campaign` remains `None` as a dormant contract field.
- `locale` defaults to `en-US` as the dormant contract field.
- The HTTP layer never passes an arbitrary raw string into the domain service as `ContextPayload`.

```
ContextPayload
├── segment
├── product_interest
├── region
├── campaign = None
└── locale = "en-US"
```

6. HTTP API Contract

The endpoint remains:

```
GET /api/onboarding/ui
```

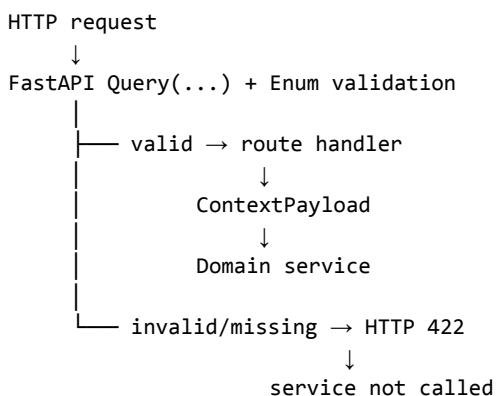
The request now requires:

```
GET /api/onboarding/ui?segment=<segment>&product_interest=<product_interest>&region=<region>
```

The response remains the `UISpec` itself. No new response envelope was introduced.

7. FastAPI Boundary Validation

FastAPI validates each query parameter against its corresponding Enum before the route handler is entered.



This prevents invalid context from being silently converted to a default value. For example, an unknown segment does not become individual.

8. Domain Service Flow

The service signature is now conceptually:

```
get_landing_ui_spec(context: ContextPayload) -> dict
```

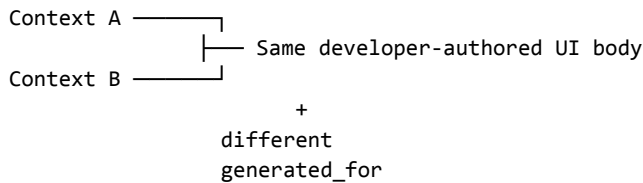
Its sequence is:

1. Deep-copy the developer-authored fixture.
2. Overwrite meta.generated_for using the supplied ContextPayload.
3. Pass the UISpec and the same context to the existing deterministic validator.
4. Return the validated UISpec or raise the existing invalid-fixture error path.

```
ContextPayload
  ↓
deep-copy developer fixture
  ↓
meta.generated_for = request context
  ↓
domain.validator.validate(spec, context)
  ↓
approved UISpec
```

9. Why the Fixture Does Not Dynamically Generate UI

Task 6A deliberately uses the controlled fixture strategy where the body, structure, and component choices remain developer-authored and constant. Only meta.generated_for is retargeted to the request context.



This is important because context-driven content generation is not being confused with model-driven generation. The runtime model has not been introduced yet.

10. Deterministic Validator Integration

The existing validator remains the single deterministic validation boundary. Task 6A does not duplicate validation rules in Python or TypeScript.

- The complete ContextPayload shape is supplied to the validator.
- The validator checks that meta.generated_for corresponds to the request context.
- The JSON Schema remains unchanged.
- domain/validator.py remains unchanged.

Trust boundary: The application does not render an arbitrary response merely because the API returned HTTP 200. The approved UISpec still has to pass the established validation and rendering path.

11. React API Client Changes

The React client now requires ContextPayload when requesting the onboarding UISpec.

```
fetchOnboardingUISpec(context)
```

↓

```
buildOnboardingUiUrl(context)
```

↓

```
URLSearchParams
```

↓

```
/api/onboarding/ui?segment=...&product_interest=...&region=...
```

URLSearchParams is used rather than manual string concatenation. This ensures special characters are encoded safely.

A client-side assertContextEcho guard also checks that the response's meta.generated_for matches the requested context.

12. React Application Changes

App.tsx currently defines a developer-controlled INITIAL_CONTEXT. It is a compile-time application setting, not a user-configurable runtime AI input.

```
INITIAL_CONTEXT = {  
  segment: "individual",  
  product_interest: "checking",  
  region: "US"  
}
```

The ready path remains deliberately simple:

```
render(state.spec)
```

The application still does not import or directly invoke the component registry. The controlled renderer remains the UI trust boundary.

13. Loading and Error Behaviour

State	Behaviour
Loading	Show loading status.
Ready	Render only the approved UISpec through <code>render(state.spec)</code> .
Error	Show an explicit error; do not silently substitute a hard-coded fixture.

14. Why No Silent Fallback?

If an API request fails because the context is invalid or the response is malformed, silently displaying a developer-authored fallback would hide the failure. The application therefore exposes the error instead.

Enterprise principle: Failures at a trust boundary should be observable rather than silently bypassed.

15. Security and Control Properties

- Only contract-defined context values are accepted.
- Invalid and missing context produces HTTP 422.
- `URLSearchParams` prevents query-parameter injection through encoding.
- Context values cannot become component names, module paths, HTML, JavaScript, or executable code.
- The existing frozen component registry remains unchanged.
- No `eval`, `Function`, `dynamic import`, `dangerouslySetInnerHTML`, or equivalent execution path was introduced.
- No AI SDK or model integration was introduced.
- The renderer trust boundary remains unchanged.

16. What Task 6A Does NOT Build

- No runtime AI model.
- No Model Gateway.
- No Agent SDK runtime integration.
- No document processing.
- No risk scoring.
- No customer approval workflow.
- No authentication subsystem.
- No database.
- No production deployment infrastructure.
- No modification to the UISpec JSON Schema.
- No modification to the controlled renderer.

17. Test Coverage

Python:

```
python -m pytest -v
55 / 55 passed
24 Task 3 validator tests
31 Task 6 / Task 6A API tests
```

React:

```
npm test
57 / 57 passed
across 7 test files
```

JSON Schema regression:

```
12 / 12 passed
```

Production build:

```
npm run build → TypeScript check passed; Vite build passed
```

18. Task 6A API Smoke Tests

Request	Status	Result
GET /api/onboarding/ui (no context)	422	Required parameters reported as missing
individual + checking + US	200	Full UISpec; generated_for matches request
corporate + loan + APAC	200	Same controlled UI body; generated_for changes
segment=unknown	422	Enum validation rejects input
region=MARS	422	Enum validation rejects input

19. Browser Smoke Test

The Vite application successfully proxied the ContextPayload request to FastAPI and received the full UISpec. The browser page exposes the actual query string for developer visibility.

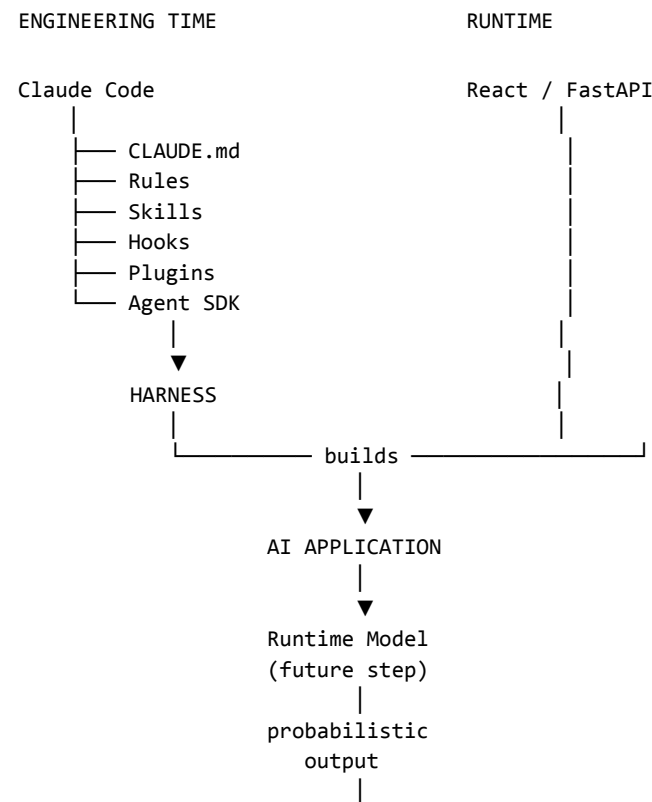
```
http://127.0.0.1:5173/
```

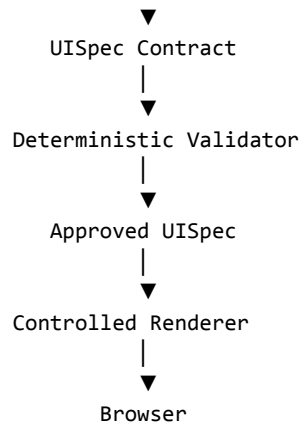
20. Architectural Invariants Confirmed

Question	Answer
Does ContextPayload enter through HTTP?	Yes — three required query parameters.
Does the Domain Runtime receive it explicitly?	Yes — <code>get_landing_ui_spec(context)</code> .
Does the validator receive the same context?	Yes — <code>validate(spec, context.as_dict())</code> .
Does <code>generated_for</code> match the request?	Yes — service retargeting + validator + client echo guard.
Does the API return a <code>UISpec</code> ?	Yes — no response envelope.
Does React render only through <code>render()</code> ?	Yes.
Can context execute arbitrary components?	No.
Has AI integration been introduced?	No.
Was the JSON Schema modified?	No.
Was the controlled renderer modified?	No.

21. Engineering-Time vs Runtime Perspective

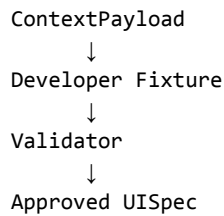
Task 6A belongs to the runtime application path, but it prepares the seam that will later connect to the probabilistic runtime model. It therefore complements the separate Claude Code engineering-time harness.



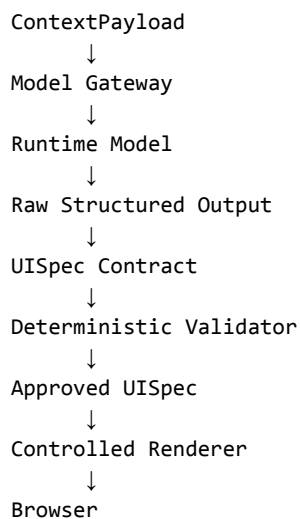


22. The Future AI Insertion Point

Task 6A gives the future model gateway an explicit input seam. The current implementation is:



The future implementation can become:



Important: The runtime model remains probabilistic. The deterministic validator and controlled renderer remain the mechanisms that constrain what the model is allowed to affect.

23. Task 6A Implementation Prompt — Exact Prompt Used

The following is the exact implementation prompt used to instruct Claude Code for Task 6A.

Task 6A — Introduce the ContextPayload request seam

Previous checkpoint:

Task 6 completed and checkpointed at commit a55c522.

Objective:

Introduce the ContextPayload request seam into the existing Customer Onboarding AI application.

Current flow:

```
React
↓
GET /api/onboarding/ui
↓
FastAPI
↓
Developer-authored UISpec fixture
↓
Deterministic validator
↓
Approved UISpec
↓
Controlled renderer
↓
Browser
```

Target flow:

```
React
↓
GET /api/onboarding/ui
  ?segment=individual
  &product_interest=checking
  &region=US
↓
FastAPI
↓
ContextPayload
↓
Developer-authored UISpec fixture
↓
Deterministic validator
↓
Approved UISpec
↓
React
↓
Controlled renderer
↓
Browser
```

Important:

Do NOT introduce AI/model integration yet.

The purpose of this task is to establish the explicit context seam that the future model gateway will consume.

Before changing anything:

Customer Onboarding AI — Task 6A | ContextPayload Request Seam

1. Inspect the repository.
2. Read the existing implementation and relevant documentation from Tasks 1-6.
3. Read:
 - CLAUDE.md
 - docs/business-problem.md
 - docs/architecture.md
 - specs/spec.md
 - specs/model-contracts.md
 - specs/schemas/ui-spec.schema.json
 - domain/validator.py
 - api/main.py
 - api/routes/onboarding.py
 - api/services/onboarding.py
 - api/fixtures/onboarding_ui.py
 - ui/src/api/client.ts
 - ui/src/App.tsx
4. Treat the existing ContextPayload contract as authoritative.

ContextPayload contract:

The request context must contain exactly these externally supplied fields:

- segment
- product_interest
- region

Use the exact allowed values already defined by the existing model contract.

Do NOT silently broaden the accepted value sets.

Expected values are:

segment:

- individual
- small-business
- corporate

product_interest:

- checking
- savings
- credit-card
- loan
- unspecified

region:

- US
- EU
- UK
- APAC

The model contract also contains dormant context fields where applicable.

Do not redesign the model contract.

Do not modify specs/model-contracts.md.

Do not modify specs/schemas/ui-spec.schema.json unless absolutely necessary to resolve an actual existing inconsistency. If such an inconsistency is discovered, stop and report it rather than redesigning the contract.

Implementation requirements:

1. Add a small Python context module.

Create:

api/context.py

It should define:

- Segment
- ProductInterest
- Region

as string Enum classes matching the model contract.

Also define a ContextPayload dataclass.

The dataclass should:

- contain the three externally supplied fields
- represent any dormant contract fields using their existing contract defaults
- be immutable/frozen where practical
- provide a small helper for converting itself to the dictionary shape expected by the validator
- provide a helper for constructing meta.generated_for

Do not add unnecessary abstractions.

2. Update the FastAPI onboarding route.

Change:

GET /api/onboarding/ui

so it requires:

```
segment
product_interest
region
```

as query parameters.

Use FastAPI's typed query validation with the existing enum values.

Invalid or missing context must produce HTTP 422.

Do not silently default invalid values.

Do not enter the domain service when FastAPI rejects the request.

The response body must remain the UISpec itself.

Do NOT introduce a response envelope such as:

```
{
  "data": {...}
}
```

3. Update the domain service.

Change the service from:

```
get_landing_ui_spec()
```

to conceptually:

```
get_landing_ui_spec(context: ContextPayload)
```

The service must:

- receive ContextPayload explicitly
- make a defensive/deep copy of the developer-authored fixture
- ensure meta.generated_for corresponds to the supplied context
- pass the same context to the existing deterministic validator
- return the validated UISpec
- preserve existing invalid-fixture behavior

Do not introduce ambient state.

Do not introduce a repository.

Do not introduce dependency injection unless genuinely required.

Keep the service small.

4. Fixture strategy.

Do not create a dynamic UI generation system.

For this task, use a controlled fixture strategy.

The developer-authored fixture body, structure, and component choices should remain deterministic and explicitly controlled.

Context may affect only the contract-required generated_for metadata unless the existing specification already defines another context-driven behavior.

Do not add arbitrary context-driven component selection.

Do not add a rules engine.

Do not add a personalization engine.

5. Validator integration.

Use the existing domain validator.

Do not duplicate validation rules in the route.

Do not duplicate validation rules in TypeScript.

The validator must receive the same context represented in ContextPayload.

The validator remains the deterministic trust boundary.

6. Update the React API client.

Change the client so that:

Customer Onboarding AI — Task 6A | ContextPayload Request Seam

`fetchOnboardingUiSpec(...)`

requires a `ContextPayload`.

Add a corresponding TypeScript `ContextPayload` type using the same externally supplied values.

Build the query string using `URLSearchParams`.

Do NOT manually concatenate query strings.

Ensure values are safely URL encoded.

The client should continue to:

- handle HTTP failures
- handle invalid JSON
- validate the response shape

Additionally, add a small defence-in-depth check that `meta.generated_for` in the response matches the context that was requested.

Do not replace server-side validation with client-side validation.

7. Update `App.tsx`.

Introduce a developer-authored initial context, for example:

```
segment: individual
product_interest: checking
region: US
```

This is NOT runtime AI.

This is NOT user-configurable yet.

Pass it to `fetchOnboardingUiSpec`.

Keep the existing loading/error/ready state model.

The ready path must still be:

```
render(state.spec)
```

Do not import `componentRegistry` into `App.tsx`.

Do not construct React components directly in `App.tsx`.

Do not bypass the controlled renderer.

8. Error behavior.

Preserve explicit failure behavior.

If the request fails:

- show an error

- do not silently substitute the old hard-coded fixture
- do not silently repair the context
- do not silently choose a different segment/product/region

9. Tests.

Add deterministic tests covering at minimum:

Python:

- valid segment values
- valid product_interest values
- valid region values
- invalid segment
- invalid product_interest
- invalid region
- missing context parameters
- ContextPayload construction
- service receives context explicitly
- generated_for equals request context
- deterministic fixture behavior
- no input mutation
- invalid fixture still fails
- no AI SDK imported

React:

- query string construction
- URL encoding safety
- fetch success
- HTTP failure
- invalid JSON
- response shape validation
- context echo validation
- renderer path remains unchanged

The tests must demonstrate that different contexts do not cause arbitrary UI components or executable code to be selected.

10. Security constraints.

Do not introduce:

- eval
- new Function
- dynamic import
- dangerouslySetInnerHTML
- arbitrary HTML execution
- shell execution
- generated JavaScript
- generated React code
- dynamic component imports
- model SDKs
- authentication
- database access

Context values must remain data.

They must never become:

- component names
- module paths
- executable code

- HTML
- JavaScript
- shell commands

11. Dependencies.

Do not add new dependencies unless absolutely necessary.

Use the existing FastAPI/Pydantic validation capability.

Use native URLSearchParams/fetch on the React side.

12. Documentation.

Update:

- README.md
- docs/architecture.md
- tests/README.md
- ui/README.md

Do NOT update docs/walkthrough.md in this task.

The walkthrough will be updated separately after the implementation checkpoint.

13. Verification.

Run:

- Python tests
- React tests
- JSON Schema regression checks
- TypeScript/build
- API smoke tests
- browser smoke test
- security regression checks

Verify:

GET /api/onboarding/ui

without context returns 422.

Verify:

GET /api/onboarding/ui?segment=individual&product_interest=checking®ion=US

returns 200 and a valid UISpec.

Verify another valid context, such as:

```
segment=corporate
product_interest=loan
region=APAC
```

returns a valid UISpec whose meta.generated_for matches that request.

Verify invalid values return 422.

Verify the response is still the UISpec itself.

Verify React still renders only through:

```
render(approvedUISpec)
```

14. Scope boundary.

Do NOT build:

- runtime AI model
- Model Gateway
- Agent SDK runtime integration
- document processing
- customer registration
- risk AI
- approval workflow
- production infrastructure
- authentication
- database
- AI evals
- feedback loop

Those belong to later tasks.

15. Git.

Do NOT commit.

Do NOT push.

Do NOT start Task 7.

Leave the working tree available for human review.

At the end, report:

1. files created
2. files modified
3. ContextPayload design
4. exact supported context values
5. API request format
6. FastAPI validation behavior
7. domain service changes
8. fixture strategy
9. validator integration
10. React API-client changes
11. React application changes
12. loading/error behavior
13. Python tests added
14. React tests added
15. full Python test result
16. full React test result
17. JSON Schema regression result
18. production build result
19. API smoke-test result
20. browser smoke-test result
21. security regression result
22. dependency changes
23. documentation changes
24. architectural answers to these questions:

- Does ContextPayload now enter through the HTTP API?
- Does the Domain Runtime receive ContextPayload explicitly?
- Does the existing validator receive the same ContextPayload?
- Does meta.generated_for match the requested context?
- Does the API still return a UISpec rather than an envelope?
- Does React still render only through render(approvedUISpec)?
- Can any context value cause arbitrary component execution?
- Has any AI/model integration been introduced?
- Has the existing model contract or JSON Schema been modified?
- Has the controlled renderer been modified?

25. Git diff summary

26. any architectural issues discovered

27. recommended next smallest task

Stop after the report.

Do not proceed beyond Task 6A.

24. Human Review Checklist Before Checkpoint

- Confirm the diff contains only the intended Task 6A changes.
- Confirm specs/model-contracts.md and specs/schemas/ui-spec.schema.json are unchanged.
- Confirm the controlled renderer and component registry are unchanged.
- Confirm no AI/model SDK was introduced.
- Confirm invalid context returns HTTP 422.
- Confirm valid context is echoed in meta.generated_for.
- Confirm the response remains the UISpec itself.
- Confirm React still reaches the UI only through render(state.spec).
- Confirm all deterministic tests and smoke tests pass.
- Confirm no generated artifacts are left in the repository.
- Confirm no commit or push has occurred before human review.

25. Recommended Checkpoint

Once the human review is complete, Task 6A can be checkpointed in the same manner as the earlier tasks: cold-state verification, final diff review, commit, push, and confirmation that origin/main and the working tree are synchronized.

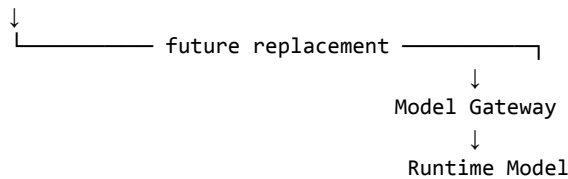
Recommended checkpoint commit:

Task 6A: Introduce ContextPayload request seam

26. What Comes Next

The next engineering increment should establish the named AI client/model seam without yet changing the observable application behavior.





This allows the application to learn the interface before probabilistic generation is introduced. The later runtime AI flow will still terminate at the existing UISpec validator and controlled renderer.

27. Final Beginner Mental Model

Task 6

We created the road.

Task 6A

We added the context lane.

Future AI

The model will eventually drive that lane.

But the model will NOT directly drive the UI.

Future trust chain

ContextPayload



Runtime Model



UISpec Contract



Deterministic Validator



Approved UISpec



Controlled Renderer



Browser

Core enterprise principle: Probabilistic AI should be introduced behind explicit interfaces and deterministic control boundaries, not allowed to bypass the application's established contracts.